# cse428-sec02-23341130

November 19, 2023

[78]:
```
NAME = "Ahmed Shakib Reza"
ID = "23341130"
COLLABORATORS_ID = ["", ""]
```

## 1 Necessary library import

[79]:
```
import numpy as np
from skimage import io, color, exposure, img_as_float
import matplotlib.pyplot as plt
```

## 2 Task 1 - Basic Image Operation

import your image or any photo taken by you (`sample.jpeg`) as a numpy array, save it in the varaible `I`.

A picture taken from your phone of any scenary/streets/building is better.

remember your image name MUST be `sample.jpeg`.

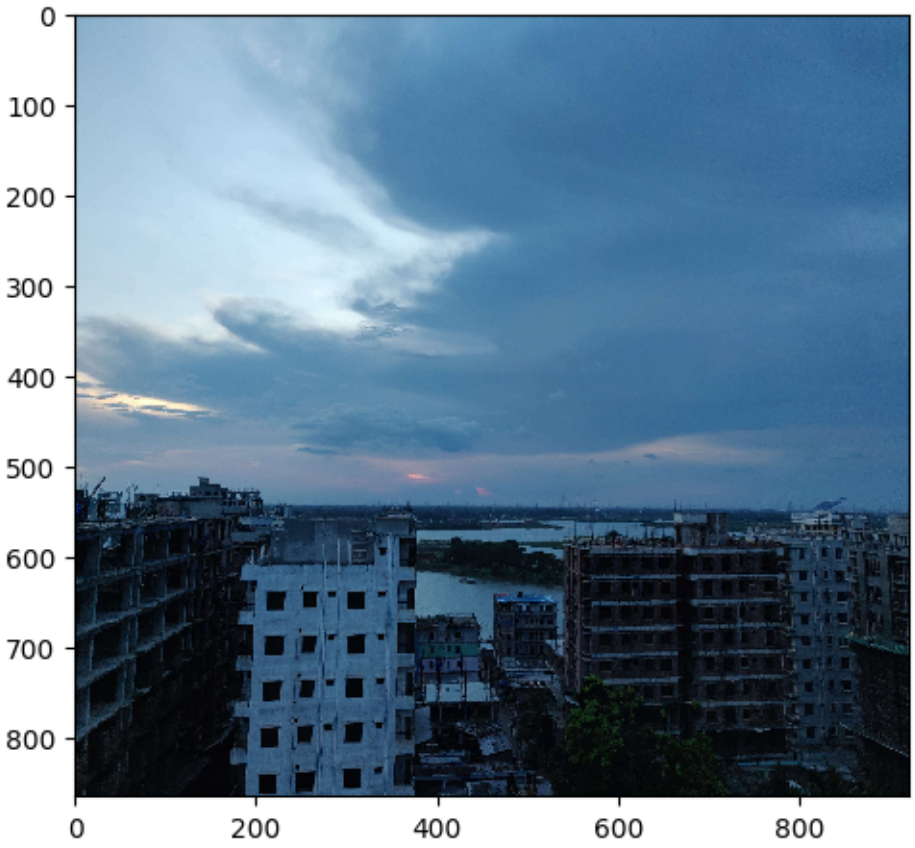
Make sure the height and the width of the image is **smaller than 1000 pixels**.

[80]:
```
I = io.imread("sample.jpeg") # Replace None with appropriate function call line


# find the height and the width of the image
H = I.shape[0] # should contain height
W = I.shape[1] # should contain width
print("Height is", H)
print("Width is", W)
### BEGIN SOLUTION
x, y = int(int(H)/1000)+1, int(int(W)/1000)+1
I = I[::x, ::y, :]
print("Updating...")
print("New Height is", I.shape[0])
print("New Width is", I.shape[1])
### END SOLUTION
```

```
Height is 3456
Width is 4608
Updating…
New Height is 864
New Width is 922
```

[81]:
```
# Normalize the image so that the gray scales are between 0 and 1. Save it to I
 ↪and display the image
#I = None

### BEGIN SOLUTION
I = img_as_float(I)

plt.figure(figsize=(5, 5))
io.imshow(I)
#plt.axis("off")
### END SOLUTION
```

[81]: <matplotlib.image.AxesImage at 0x7bef3331a110>



[82]:
```
# Increase the brightness of the image without changing the contrast.
# Save the resulting image in I_bright and display it.
I_bright = None

### BEGIN SOLUTION
I_bright = np.clip(I+0.3, 0, 1)

plt.figure(figsize=(5, 5))
io.imshow(I_bright)
plt.axis("off")
### END SOLUTION
```

[82]:
```
(-0.5, 921.5, 863.5, -0.5)
```

[83]:
```
# Decrease the brightness of the image without changing the contrast.
# Save the resulting image in I_dark and display it.
I_dark = None

### BEGIN SOLUTION
I_dark = np.clip(I-0.3, 0, 1)
```

```
plt.figure(figsize=(5, 5))
io.imshow(I_dark)
plt.axis("off")
### END SOLUTION
```

[83]: (-0.5, 921.5, 863.5, -0.5)

[84]:
```
# Multiply the three channels of image I with three DIFFERENT numbers between 0.
  ↪3 and 3
# Save the resulting image in I_tint and display it.
# The resulting image should have some color shift
I_tint = None

# HINT:
# I_tint = np.zeros(I.shape)
# I_tint[:, :, 0] =  ..... I[:, :, 0].....
# ......

### BEGIN SOLUTION

I_tint = np.zeros(I.shape)
```

```
I_tint[:, :, 0] = np.clip(I[:, :, 0] * 0.3, 0, 1)
I_tint[:, :, 1] = np.clip(I[:, :, 1] * 0.9, 0, 1)
I_tint[:, :, 2] = np.clip(I[:, :, 2] * 0.6, 0, 1)

plt.figure(figsize=(5, 5))
io.imshow(I_tint)
plt.axis("off")
### END SOLUTION
```

[84]: (-0.5, 921.5, 863.5, -0.5)

[85]:
```
# Convert the image into a grayscale image.
# Save it to I_gray and display it
I_gray = None

### BEGIN SOLUTION
I_gray = color.rgb2gray(I)

plt.figure(figsize=(5, 5))
io.imshow(I_gray)
plt.axis("off")
```

```
### END SOLUTION
```

[85]: (-0.5, 921.5, 863.5, -0.5)

[86]:
```
# Display the negative of the grayscale image

### BEGIN SOLUTION
I_neg = 1 - I_gray

plt.figure(figsize=(5, 5))
io.imshow(I_neg)
plt.axis("off")
### END SOLUTION
```

[86]: (-0.5, 921.5, 863.5, -0.5)

[87]:
```
# Artificially degrade the **grayscale image** by reducing it contrast
# You can do so by recaling the gray values and concentrating them in a narrow␣
 ↪range,
# say between 0.3 and 0.6.
# Save the image as I_degraded and display it
# HINT: SEE lec-4-demo-codes

I_degraded = None

### BEGIN SOLUTION
I_degraded = exposure.rescale_intensity(I_gray, in_range=(0, 1), out_range=(0.
 ↪3, 0.6))

plt.figure(figsize=(5, 5))
io.imshow(I_degraded)
plt.axis("off")
### END SOLUTION
```

[87]:
```
(-0.5, 921.5, 863.5, -0.5)
```

[88]:
```
# Complete the following function to perform Piecewise Linear Contrast stretching
# That is, implement the map shown in Slide 17 of Lecture 3

# Prototype: piecewise_contrast_stretch(I_gray, r1, r2, s1, s2)
# Assuming both input and output images are normalized between 0 and 1

def piecewise_contrast_stretch(I, r1, r2, s1, s2):
    pass
    # Write your code here
    I_stretched = np.zeros(I_degraded.shape)

    ### BEGIN SOLUTION
    alpha = (s1-0)/(r1-0)
    beta = (s2-s1)/(r2-r1)
    gamma = (1-s2)/(1-r2)

    for i in range(I_degraded.shape[0]):
      for j in range(len(I_degraded[i])):
        r = I_degraded[i][j]
        if r>=0 and r<r1:
```

```
            I_stretched[i][j] = np.clip(alpha*r, 0, 1)
        elif r>=r1 and r<r2:
            I_stretched[i][j] = np.clip(beta*(r-r1) + s1, 0, 1)
        elif r>=r2 and r<1:
            I_stretched[i][j] = np.clip(gamma*(r-r2) + s2, 0, 1)


    return I_stretched
    ### END SOLUTION
```

[89]:
```
# To test your implementation, contrast strech the degraded image I_degrade
r1 = 0.3
r2 = 0.6
s1 = 0.05
s2 = 0.95
I_stretched = piecewise_contrast_stretch(I_degraded, r1, r2, s1, s2)

# Display the strtched image

### BEGIN SOLUTION
plt.figure(figsize=(5, 5))
io.imshow(I_stretched)
plt.axis("off")
### END SOLUTION
```

[89]:
```
(-0.5, 921.5, 863.5, -0.5)
```

## 3 Task 2 - Histogram and Equalization

[90]:
```
# Plot the Image and its histogram + cdf of the original image I
# Note that it is a color image, so it will have three different histograms

### BEGIN SOLUTION

def plot_hist_and_cdf(I, nbins=256, normalize=False, plot_cdf=True):
    hist, bins_hist = exposure.histogram(I.ravel(), nbins=nbins,␣
 ↪normalize=normalize)
    plt.plot(bins_hist, hist, 'k')
    plt.xlabel("pixel values")
    if normalize:
        plt.ylabel("probability")
    else:
        plt.ylabel("count")
    xmax = 1 if I.max() <= 1 else 255
    plt.xlim([0, xmax])

    if plot_cdf:
```

```
        cdf, bins_cdf = exposure.cumulative_distribution(I.ravel(), nbins=nbins)
        plt.twinx()
        plt.plot(bins_cdf, cdf, 'r', lw=3)
        plt.ylabel("percentage")
    plt.tight_layout()
def plot_img_hist_cdf(I, nbins=256, normalize=False, plot_cdf=True, figsize=(8,␣
 ↪12)):

  if len(I.shape) == 3:  # for color image
    fig, ax = plt.subplots(4, 1, figsize=figsize)
    plt.subplot(4, 1, 1)
    io.imshow(I)
    plt.axis("off")
    plt.title("image")
    clr = ["Red", "Green", "Blue"]
    clr1 = ["red", "green", "blue"]
    for i in range(2,5):
      plt.subplot(4, 1, i)
      plot_hist_and_cdf(I[:, :, i-2])
      plt.title(clr[i-2], color=clr1[i-2])
  else: # for black and white image
    fig, ax = plt.subplots(2, 1, figsize=figsize)
    plt.subplot(2, 1, 1)
    io.imshow(I)
    plt.axis("off")
    plt.title("image")
    plt.subplot(2, 1, 2)
    plot_hist_and_cdf(I)
  plt.tight_layout()



plot_img_hist_cdf(I)

### END SOLUTION
```

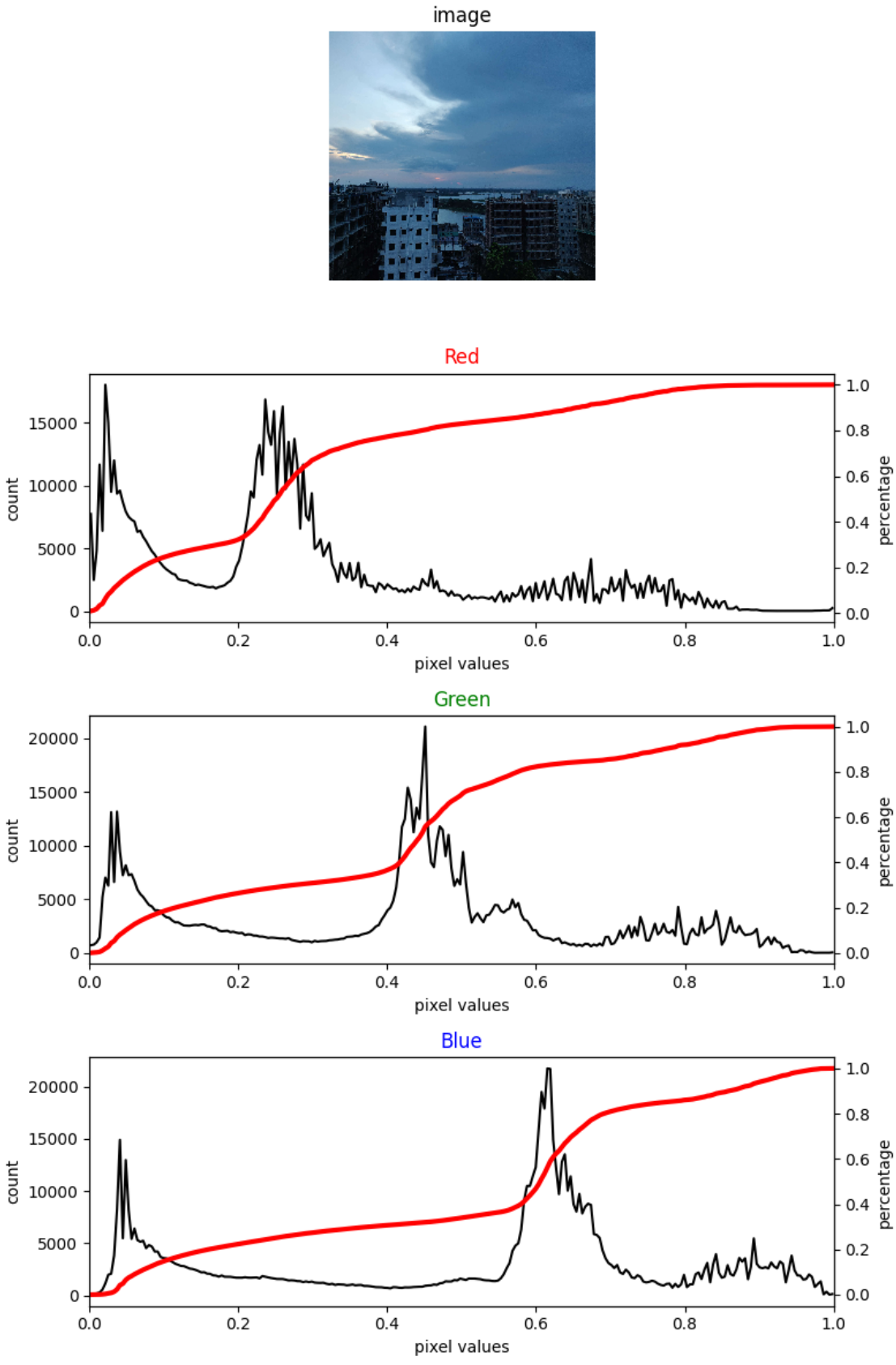
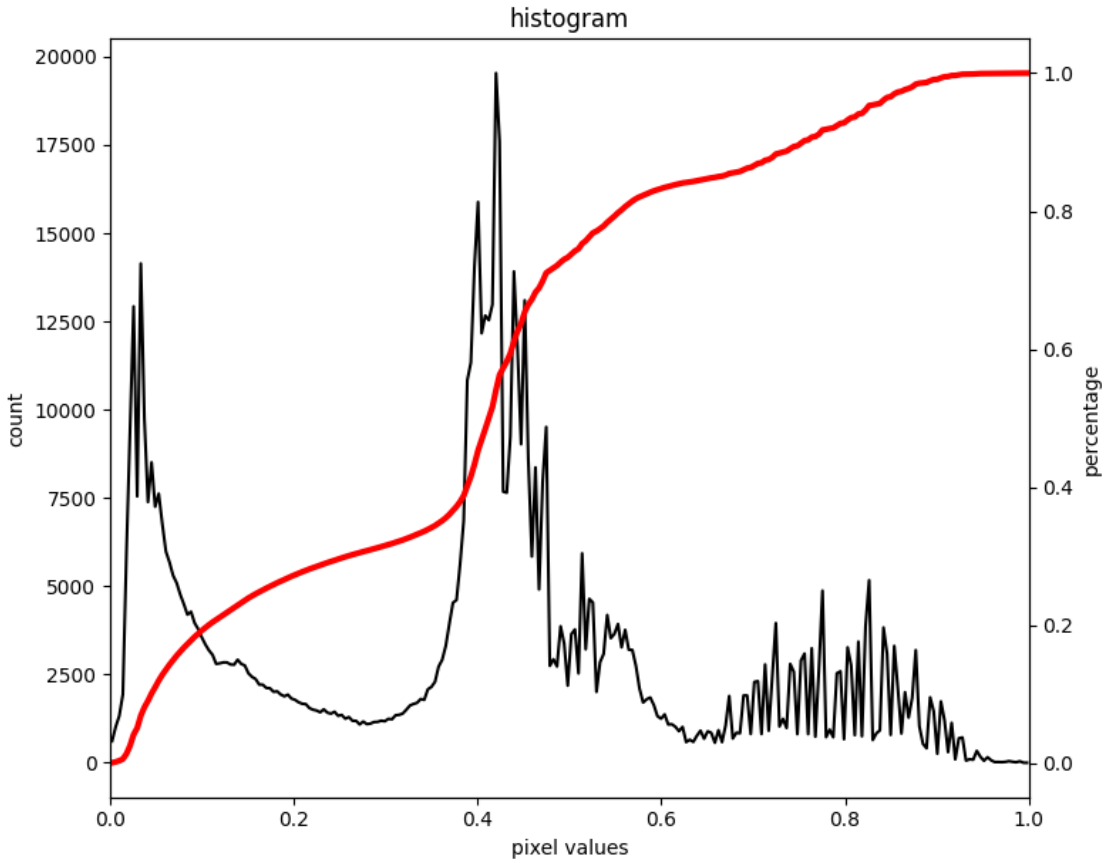


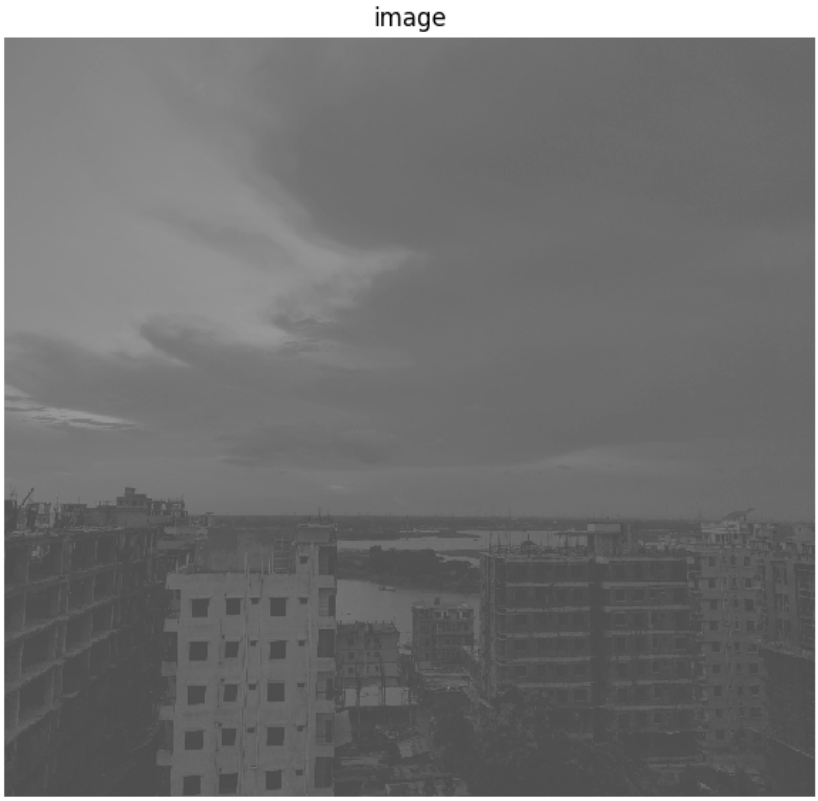
image
Red
count
percentage
pixel values
Green
count
percentage
pixel values
Blue
count
percentage
pixel values

[91]:
```
# Plot the Image and its histogram + cdf of the grayscale image I_gray

### BEGIN SOLUTION
plot_img_hist_cdf(I_gray)
plt.title("histogram")
plt.tight_layout()
### END SOLUTION
```

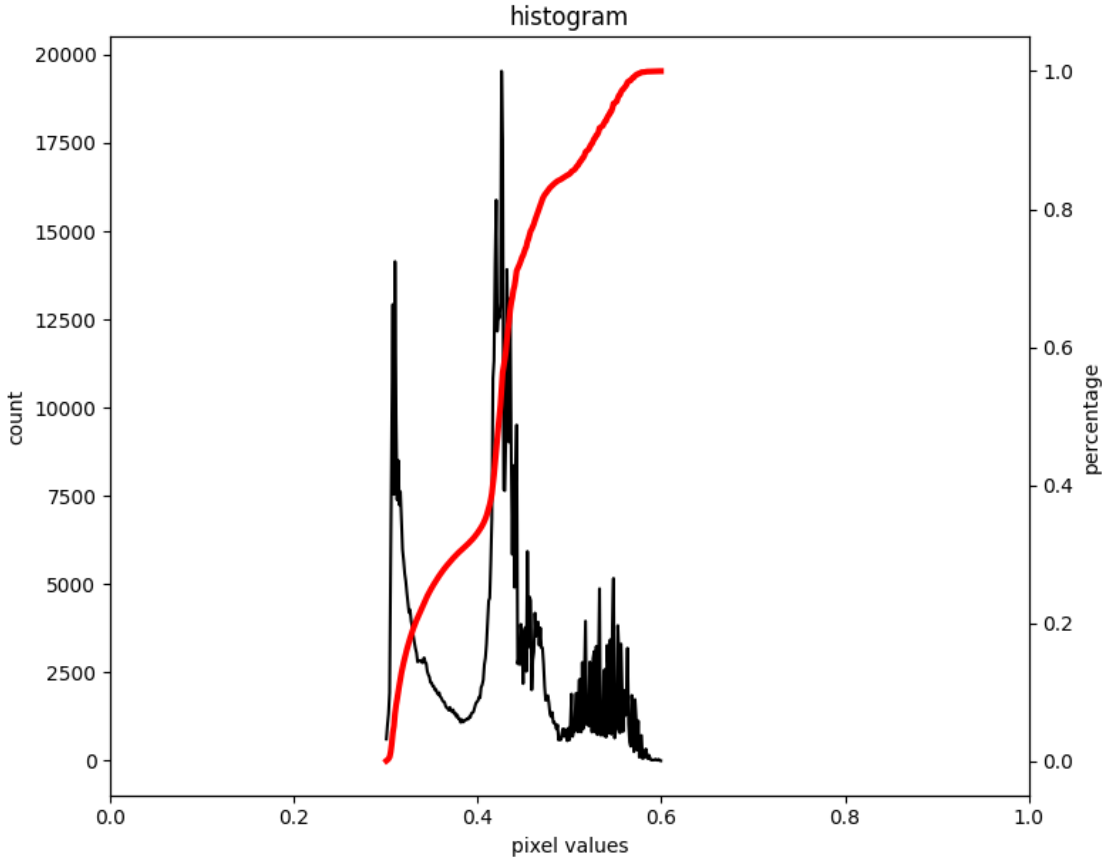

image


histogram
count
percentage
pixel values
0
2500
5000
7500
10000
12500
15000
17500
20000
0.0
0.2
0.4
0.6
0.8
1.0

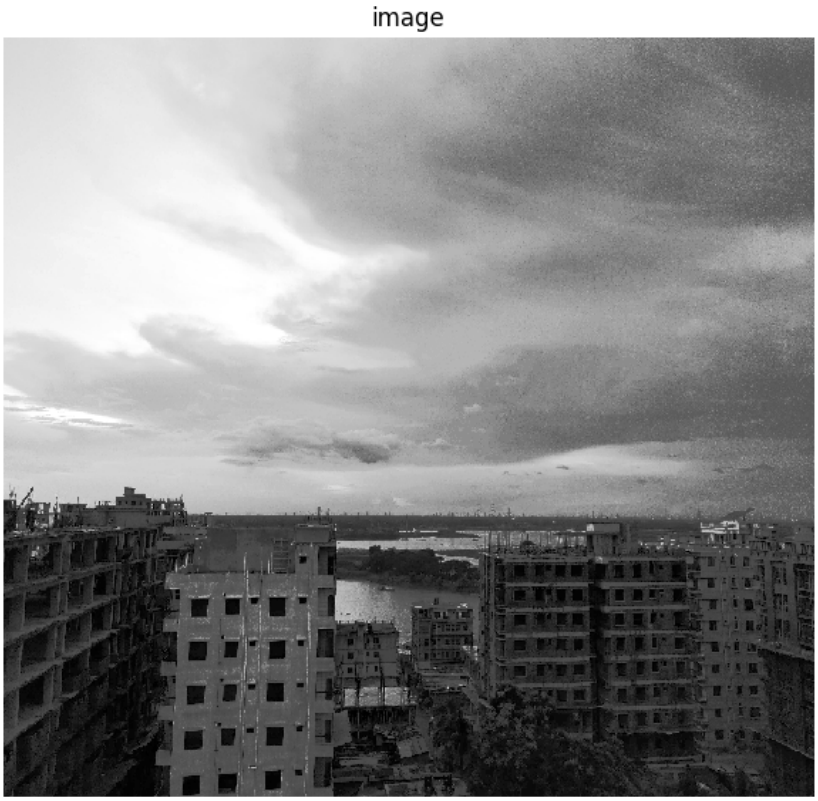
[92]:
```
# Plot the Image and its histogram + cdf of the degraded image I_degraded

### BEGIN SOLUTION
plot_img_hist_cdf(I_degraded)
plt.title("histogram")
plt.tight_layout()
### END SOLUTION
```



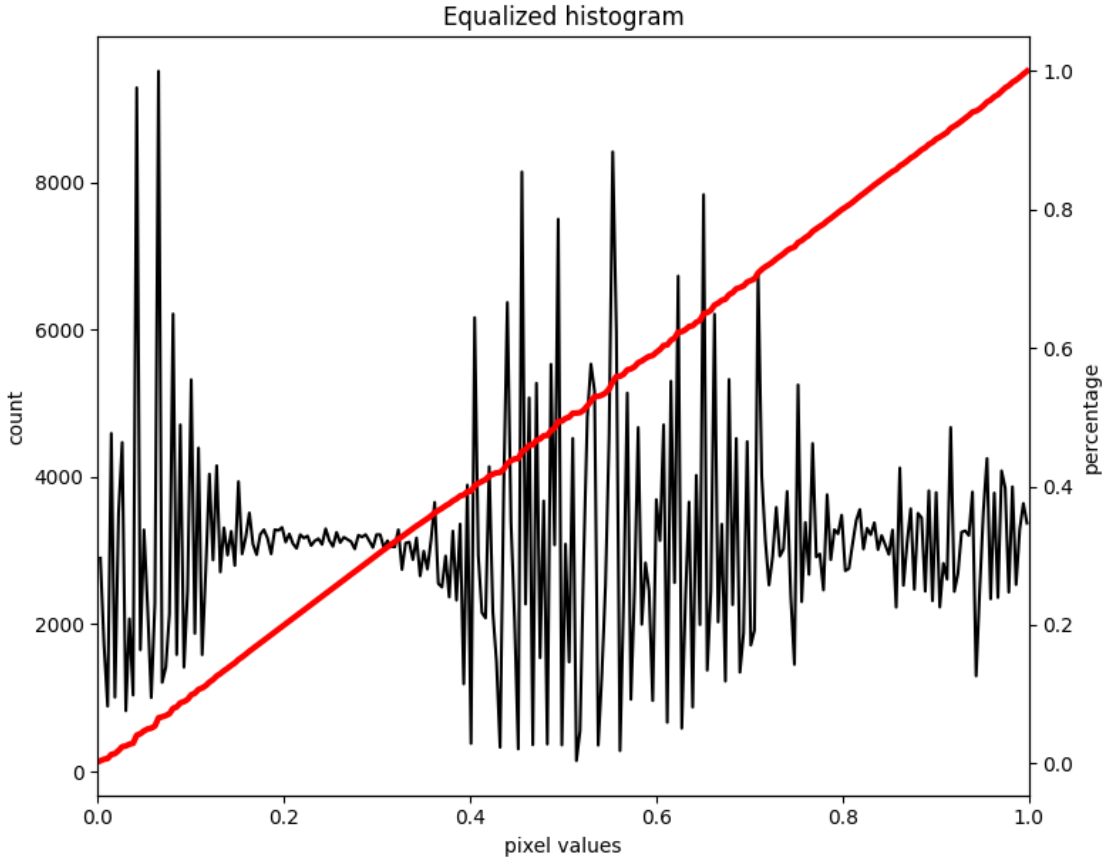
image


histogram
count
percentage
pixel values
20000
17500
15000
12500
10000
7500
5000
2500
0
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

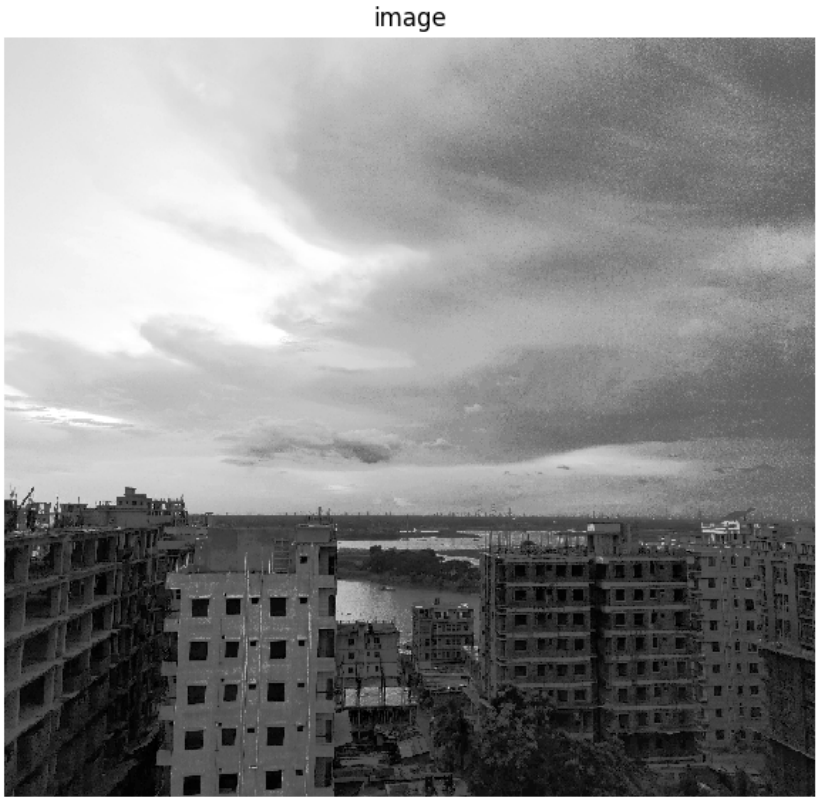
[93]:
```
# Equalize the histogram of the degraded image I_degraded
# Save the result in I_recon_gray, display the image along with its histogram

I_recon_gray = None

### BEGIN SOLUTION

I_recon_gray = exposure.equalize_hist(I_degraded)

plot_img_hist_cdf(I_recon_gray)
plt.title("Equalized histogram")
plt.tight_layout()

### END SOLUTION
```



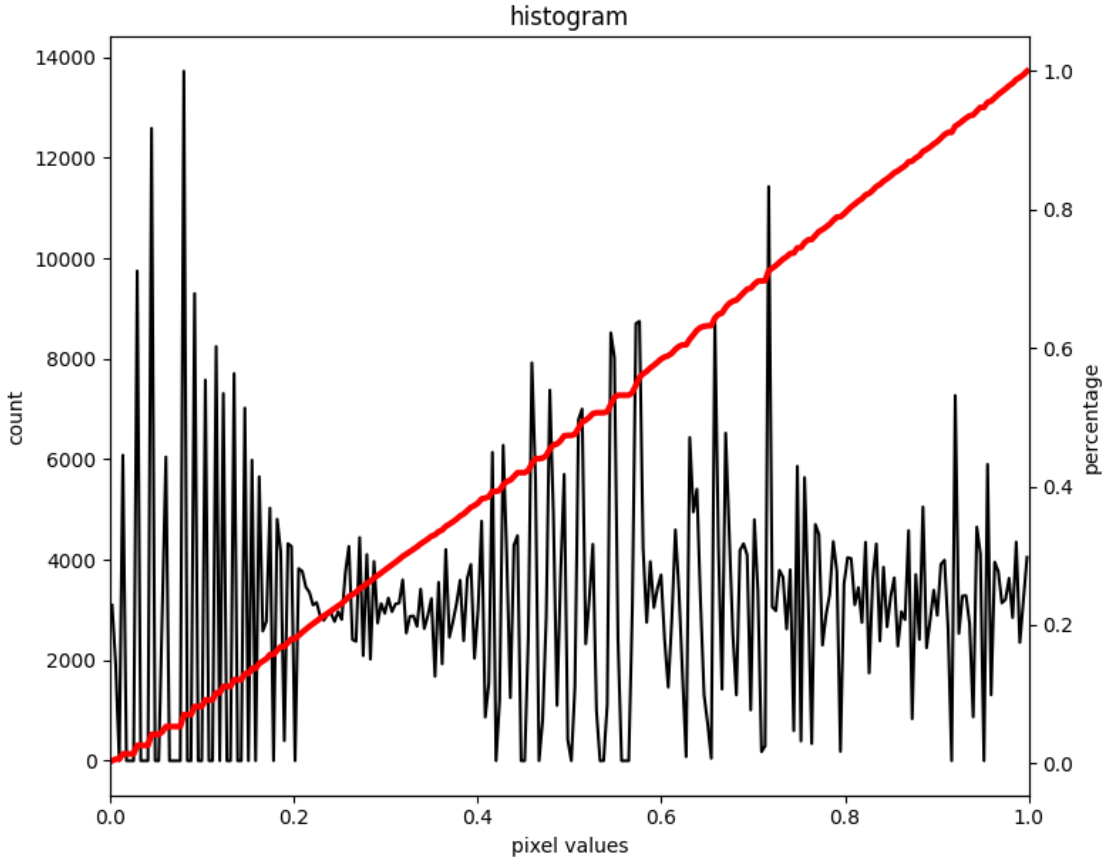
image


Equalized histogram
count
percentage
pixel values
0
2000
4000
6000
8000
0.0
0.2
0.4
0.6
0.8
1.0

[94]:
```
# Equalize the histogram of the degraded image I_degraded using AHE
# Save the result in I_recon_gray_2, display the image along with its histogram

#I_recon_gray_2 = None

### BEGIN SOLUTION
I_recon_gray_2 = exposure.equalize_adapthist(I_degraded, kernel_size=(900,␣
 ↪900), clip_limit=0) # kernel_size=(900, 900) gives the best reault

plot_img_hist_cdf(I_recon_gray_2)
plt.title("histogram")
plt.tight_layout()
### END SOLUTION
```



image


histogram
14000
12000
10000
8000
6000
4000
2000
0
count
0.0
0.2
0.4
0.6
0.8
1.0
pixel values
1.0
0.8
0.6
0.4
0.2
0.0
percentage

[95]:
```
# Equalize the histogram of the degraded image I_degraded using CLAHE
# Save the result in I_recon_gray_2, display the image along with its histogram

#I_recon_gray_3 = None

### BEGIN SOLUTION
I_recon_gray_3 = exposure.equalize_adapthist(I_degraded, kernel_size=(512,␣
↪512), clip_limit=0.01)

plot_img_hist_cdf(I_recon_gray_3)
plt.title("histogram")
plt.tight_layout()
### END SOLUTION
```

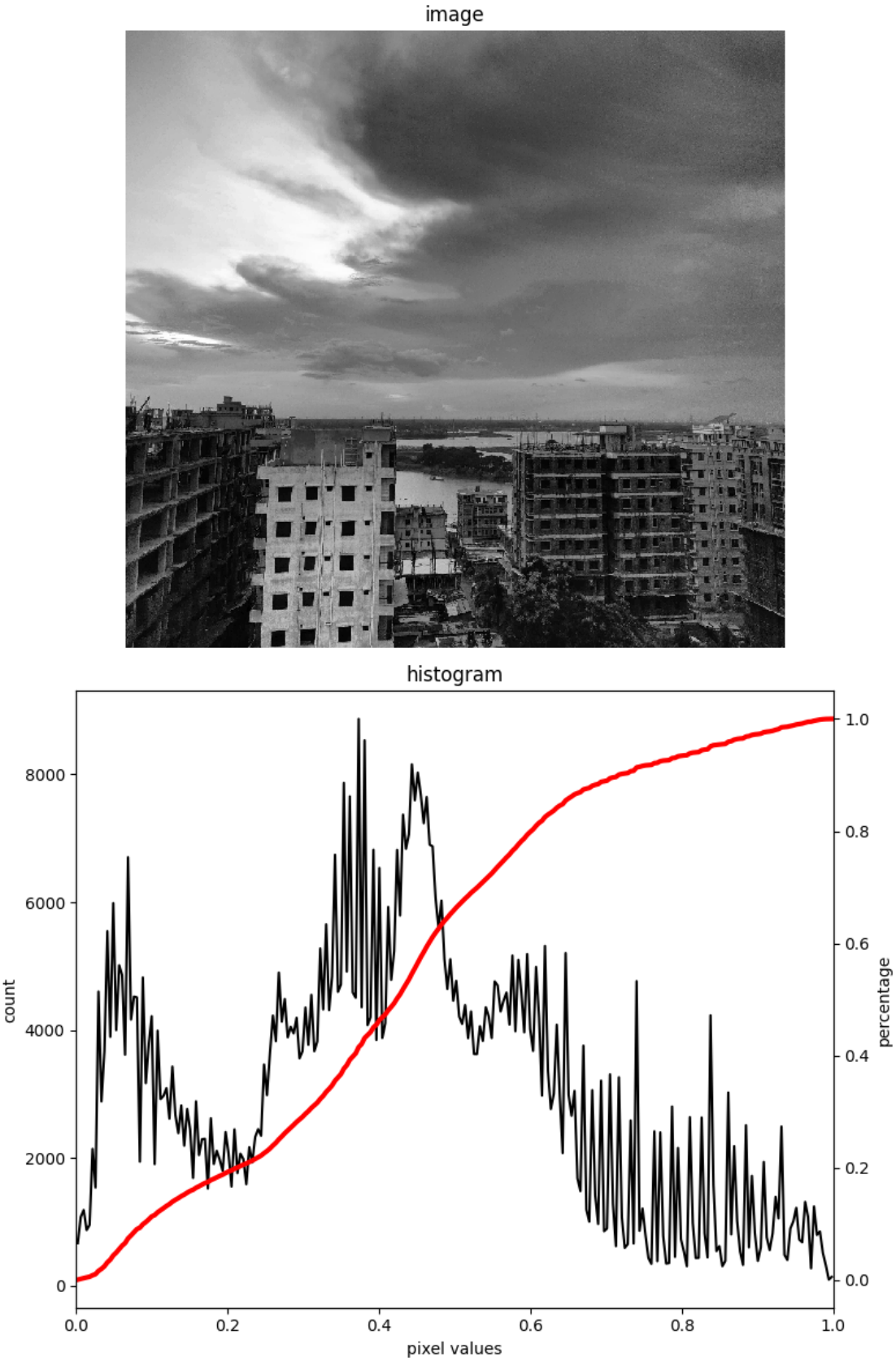


image
histogram
count
percentage
pixel values
0
2000
4000
6000
8000
0.0
0.2
0.4
0.6
0.8
1.0

[96]:
```
# Artificially degrade the original **RGB image** by reducing it contrast
# You can do so by recaling the values of the L channel (in LAB color space)
# and concentrating them in a narrow range, say between 0.3 and 0.6.
# Save the image as I_rgb_degraded and display it

I_rgb_degraded = None

### BEGIN SOLUTION

I_lab = color.rgb2lab(I)
l_channel = I_lab[:, :, 0]/100.0
scaled_l_channel = exposure.rescale_intensity(l_channel, in_range=(0, 1),␣
 ↪out_range=(0.3, 0.6))
I_lab[:, :, 0] = scaled_l_channel * 100.0
I_rgb_degraded = color.lab2rgb(I_lab)

plot_img_hist_cdf(I_rgb_degraded)
### END SOLUTION
```

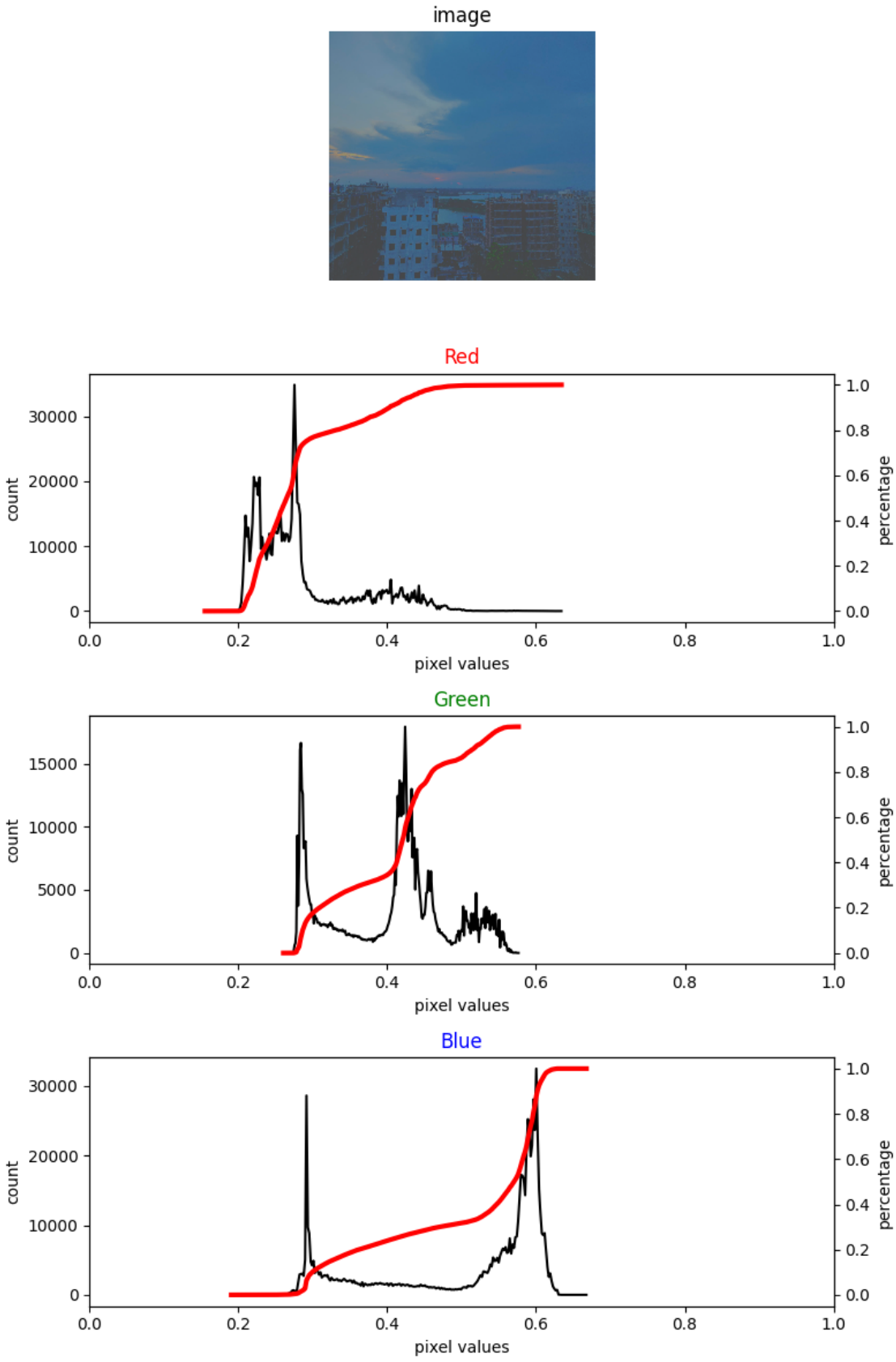


image
Red
count
percentage
pixel values
Green
count
percentage
pixel values
Blue
count
percentage
pixel values

[97]:
```
# Equalize the histogram of the degraded color image I_rgb_degraded using CLAHE
# Save the result in I_recon_color, display the image along with its histogram
# HINT: You have to convert to LAB first
# See the lecture and lecture-4-demo-codes

I_recon_color = None

### BEGIN SOLUTION

I_lab2 = color.rgb2lab(I_rgb_degraded)
l_channel = I_lab2[:, :, 0] / 100.0
clipped_l_channel = exposure.equalize_adapthist(l_channel, clip_limit=0.005)
I_lab2[:, :, 0] = clipped_l_channel * 100.0
I_recon_color = color.lab2rgb(I_lab2)


plot_img_hist_cdf(I_recon_color)
#io.imshow(I)
### END SOLUTION
```

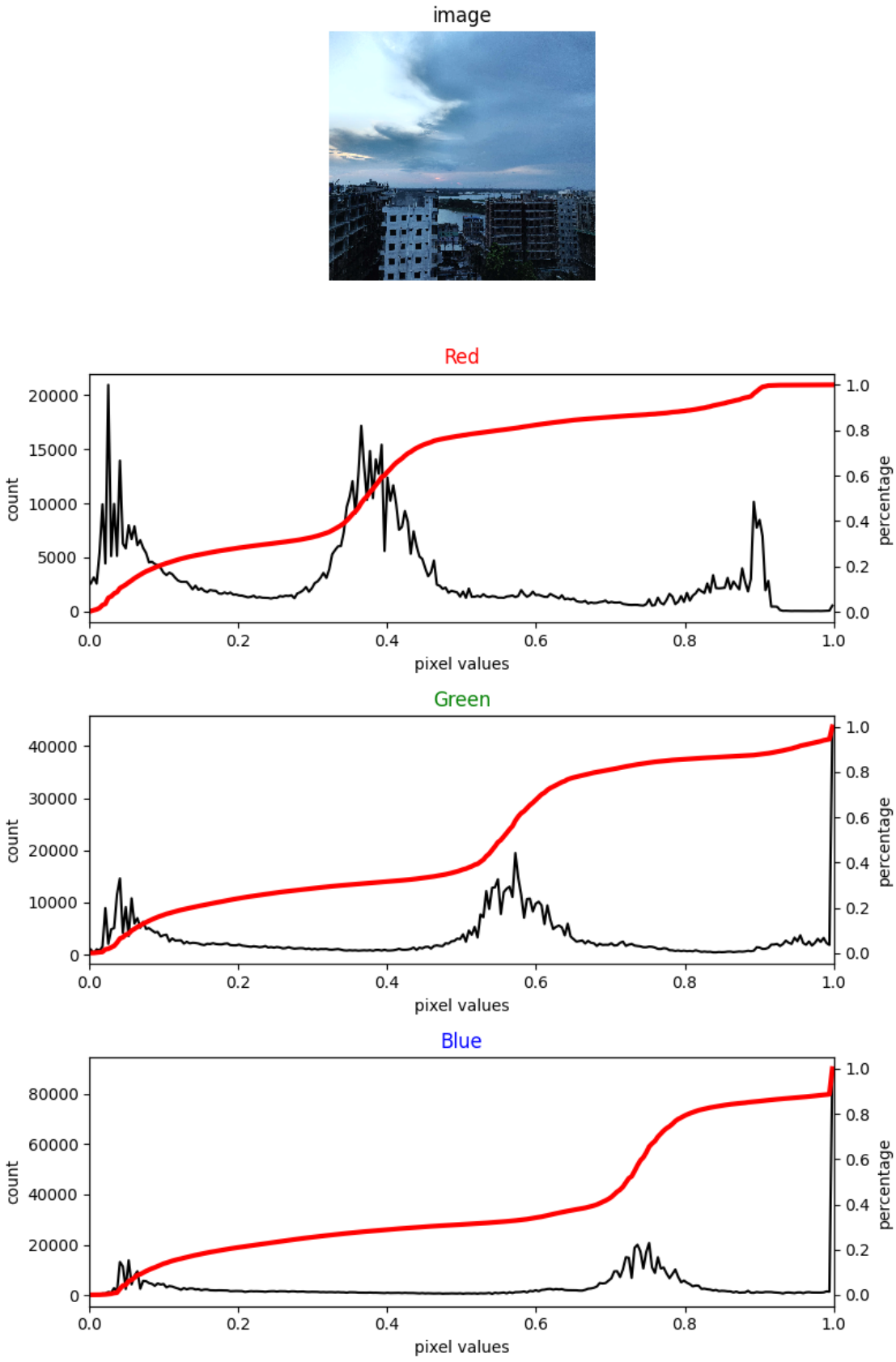


image
Red
count
percentage
pixel values
Green
count
percentage
pixel values
Blue
count
percentage
pixel values

# 4 Task 3 - Open Ended

*## There are 3 images in the drive directory below. Look at the questions from the brackets [.]. Answer them in the provided text cell at the bottom.*

### link: https://drive.google.com/drive/folders/1ft3XrO-MGxhxL2PfcLCFQjU0wvvlLAv4?usp=sharing

[98]:
```
# Dark_Room.jpg = very dark [The windows are on walls. How does the wall look␣
 ↪like?]
# Foggy_Road.jpg = washed out/foggy [How many vehicles do you think there are?]
# Read_the_code.jpg = Dark RGB Barcode [What is hidden in the Barcode?
#                                       Make it scanable, scan it and say␣
 ↪something about the hidden message.]

# Your task is to improve these images using
# contrast stretching, histogram equalization, AHE or CLAHE
# try different combination of parameter settings to see which produces the␣
 ↪best result

### BEGIN SOLUTION

### END SOLUTION
```

### 4.0.1 Your answers:

abc abc abc ... ...

[99]:
```
def conttrast_streatch(I):
  if I.size == 2:
    I = img_as_float(I)
    I_streatched = np.zeros(I.shape)
    c1 = I.min()
    c2 = 1/(I.max()-I.min())

    for i in range(I.shape[0]):
      for j in range(len(I[i])):
        I_streatched[i][j] = np.clip(c2*(I[i][j]-c1), 0, 1)
    return I_streatched

  else:
    I_lab = color.rgb2lab(I)
    l_channel = I_lab[:, :, 0] / 100.0
    cs_l_channel = np.zeros(l_channel.shape)

    c1 = l_channel.min()
```

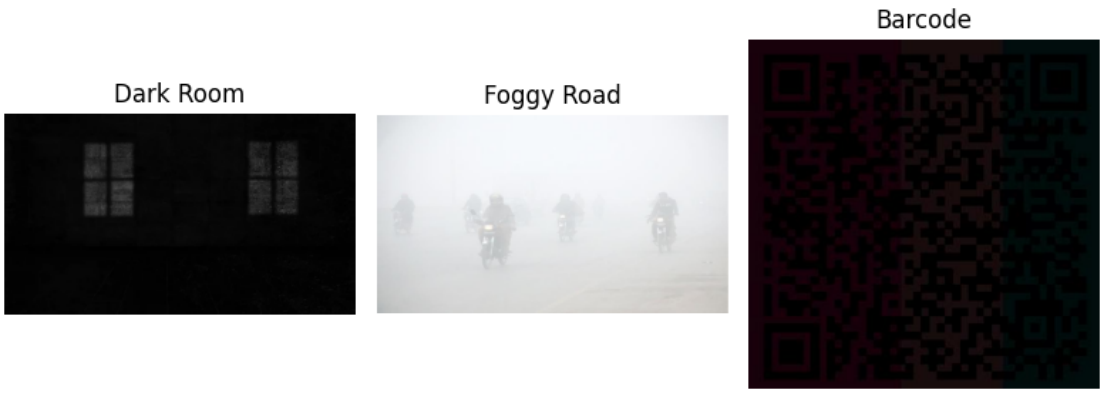
```
    c2 = 1/(l_channel.max()-l_channel.min())
    for i in range(l_channel.shape[0]):
      for j in range(len(l_channel[i])):
        cs_l_channel[i][j] = np.clip(c2*(l_channel[i][j]-c1), 0, 1)


    I_lab[:, :, 0] = cs_l_channel * 100.0
    I_streatched = color.lab2rgb(I_lab)
    return I_streatched


def HE(I):
  if I.size == 2:
    I = img_as_float(I)
    I_eq = exposure.equalize_hist(I)
    return I_eq


  else:
    I_lab = color.rgb2lab(I)
    l_channel = I_lab[:, :, 0] / 100.0
    eq_l_channel =  exposure.equalize_hist(l_channel)
    I_lab[:, :, 0] = eq_l_channel * 100.0
    I_eq = color.lab2rgb(I_lab)
    return I_eq

def AHE(I, karnel_size):
  if I.size == 2:
    I = img_as_float(I)
    I_ahe = exposure.equalize_adapthist(I, kernel_size=karnel_size,␣
 ↪clip_limit=0)
    return I_ahe
  else:
    I_lab = color.rgb2lab(I)
    l_channel = I_lab[:, :, 0] / 100.0
    eq_l_channel =  exposure.equalize_adapthist(l_channel,␣
 ↪kernel_size=karnel_size, clip_limit=0)
    I_lab[:, :, 0] = eq_l_channel * 100.0
    I_ahe = color.lab2rgb(I_lab)
    return I_ahe

def CLAHE(I, karnel_size, clip_limit):
  I = img_as_float(I)
  if I.size == 2:
    I_clahe = I_recon_gray_3 = exposure.equalize_adapthist(I,␣
 ↪kernel_size=karnel_size, clip_limit=clip_limit)
    return I_clahe
```

```
  else:
    I_lab = color.rgb2lab(I)
    l_channel = I_lab[:, :, 0] / 100.0
    clipped_l_channel = exposure.equalize_adapthist(l_channel,␣
 ↪clip_limit=clip_limit)
    I_lab[:, :, 0] = clipped_l_channel * 100.0
    I_clahe = color.lab2rgb(I_lab)

    return I_clahe


def plot_images(img_list, titles_list):

    if len(img_list) == 4:
        fig, axes = plt.subplots(2, 2, figsize=(8, 6))

        for i in range(1, 5):
            plt.subplot(2, 2, i)
            if len(img_list[i - 1].shape) == 2:
                plt.imshow(img_list[i - 1], cmap='gray')
            else:
                plt.imshow(img_list[i - 1])
            plt.axis("off")
            plt.title(titles_list[i - 1])

        plt.tight_layout()
        plt.show()

    elif len(img_list) == 3:
        fig, axes = plt.subplots(1, 3, figsize=(8, 6))

        for i in range(1, 4):
            plt.subplot(1, 3, i)
            if len(img_list[i - 1].shape) == 2:
                plt.imshow(img_list[i - 1], cmap='gray')
            else:
                plt.imshow(img_list[i - 1])
            plt.axis("off")
            plt.title(titles_list[i - 1])

        plt.tight_layout()
        plt.show()
```

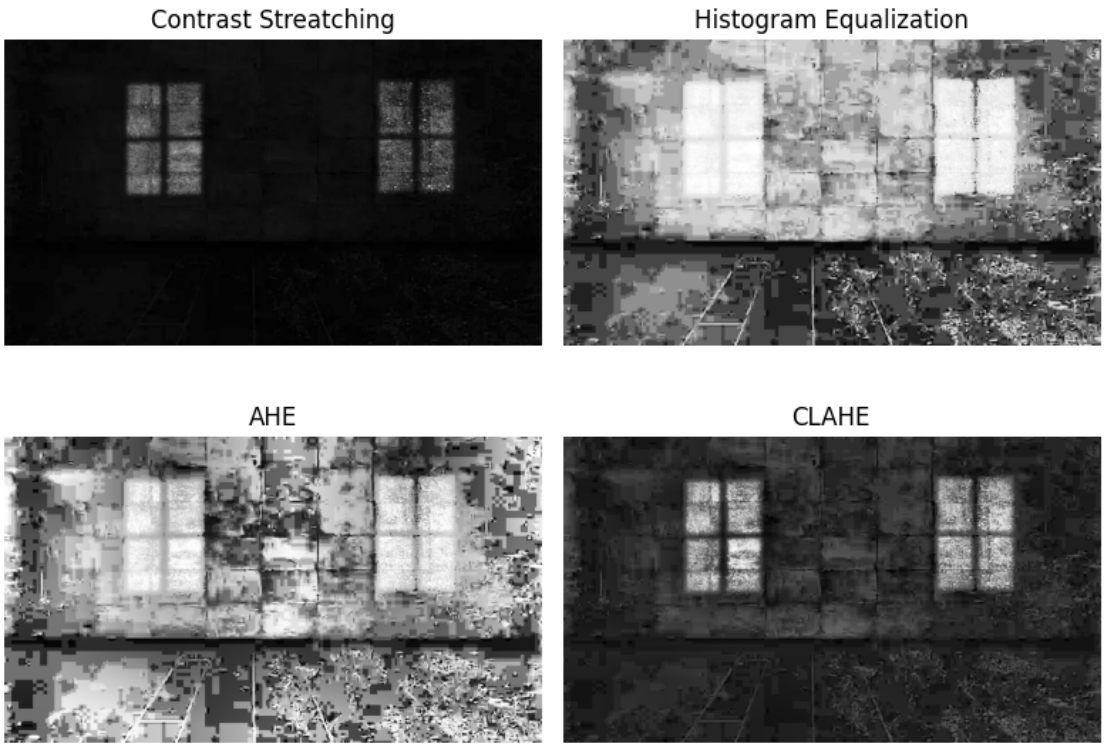
[100]:
```
I1 = io.imread("Dark_Room.jpg")
I2 = io.imread("Foggy_Road.jpg")
I3 = io.imread("Read_the_code.jpg")
```

```
original_imgs = [I1, I2, I3]
titles = ["Dark Room", "Foggy Road", "Barcode"]
```

[101]:
```
plot_images(original_imgs, titles)
```




[102]:
```
I1_CS = conttrast_streatch(I1)
I1_HE = HE(I1)
I1_AHE = AHE(I1, (128,128))
I1_CLAHE = CLAHE(I1, (128,128), 0.05)

img = [I1_CS, I1_HE, I1_AHE, I1_CLAHE]
titles2 = ["Contrast Streatching", "Histogram Equalization", "AHE", "CLAHE"]

plot_images(img, titles2)
```




- ***Question 1:*** **Dark_Room.jpg = very dark [The windows are on walls. How does the wall look like?]**

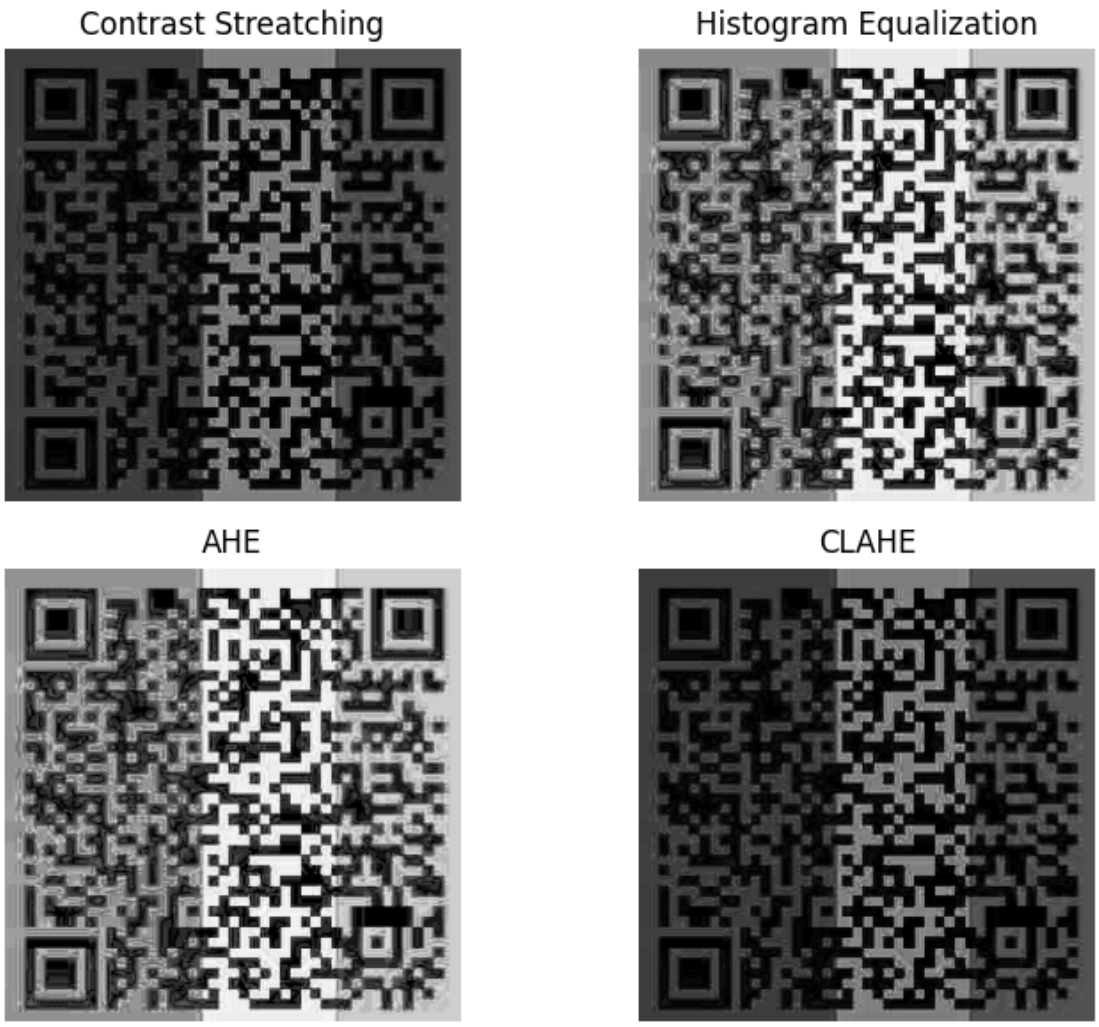
Answer: The wall is made of rectangular tiles. There are 2 rectangular windows with 4 rectangular seperations in each window, in the opposite wall and the shadows of these windows are visible on the wall of the given image.

[103]:
```
I2_CS = conttrast_streatch(I2)
I2_HE = HE(I2)
I2_AHE = AHE(I2, (64,64))
I2_CLAHE = CLAHE(I2, (0.5,0.5), 0.001)



img = [I2_CS, I2_HE, I2_AHE, I2_CLAHE]
titles2 = ["Contrast Streatching", "Histogram Equalization", "AHE", "CLAHE"]

plot_images(img, titles2)
```


- ***Question 2:*** **Foggy_Road.jpg = washed out/foggy [How many vehicles do you think there are?]**

Answer: There are 8 vehicles. 7 motorbikes and 1 car.

[104]:
```
I3_CS = color.rgb2gray(conttrast_streatch(I3))
I3_HE = color.rgb2gray(HE(I3))
I3_AHE = color.rgb2gray(AHE(I3, (800,800)))
I3_CLAHE = color.rgb2gray(CLAHE(I3, (12,12), 0.0001))



img = [I3_CS, I3_HE, I3_AHE, I3_CLAHE]
titles2 = ["Contrast Streatching", "Histogram Equalization", "AHE", "CLAHE"]

plot_images(img, titles2)
```




- ***Question 3:*** **Read_the_code.jpg = Dark RGB Barcode [What is hidden in the Barcode? Make it scanable, scan it and say something about the hidden message.]**

Answer: The barcode gives the link of CSE428 theory Practice Sheet
(https://docs.google.com/document/d/1Y3Z_DvDcCJG9aOC4N4CMfX4KM5wGP-cxBekb8WT2cfg/edit#heading=h

[104]: